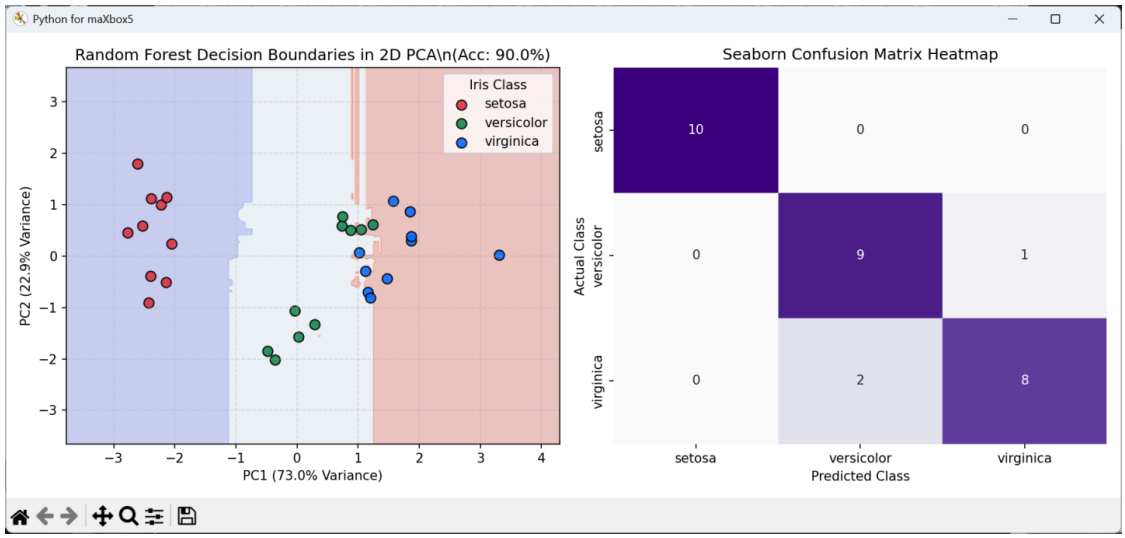
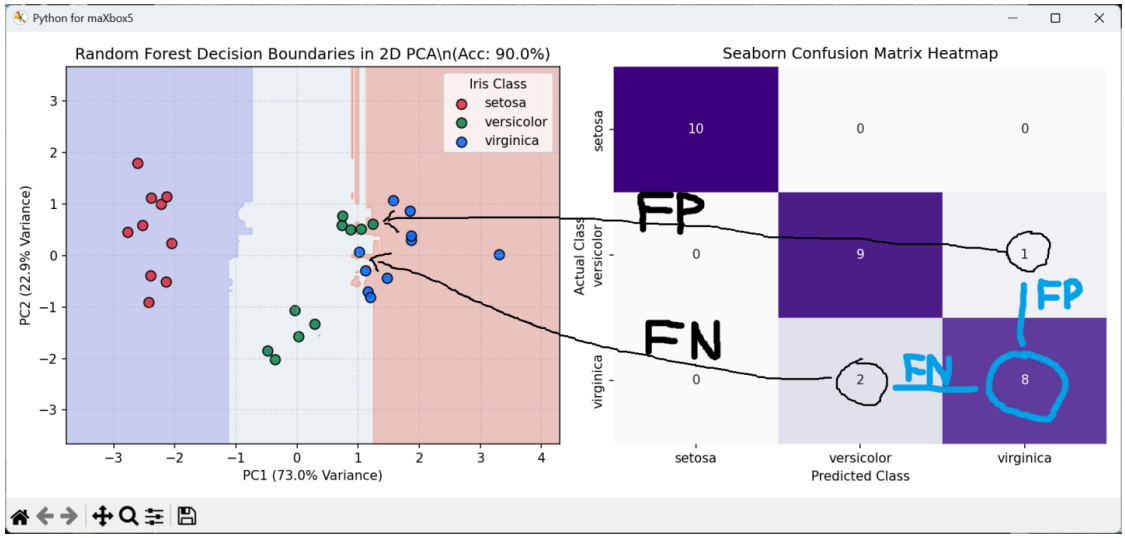


https://sourceforge.net/projects/maxbox5/files/examples/1501_wine_dataset_ml_studio23_iriscomp.pas/download



Virginica has 1 False Positive (the green ones flaged) and has 2 False Negatives (the blue ones missed)

Machine Learning Python Update

we update from 3.14.3 to 3.14.8

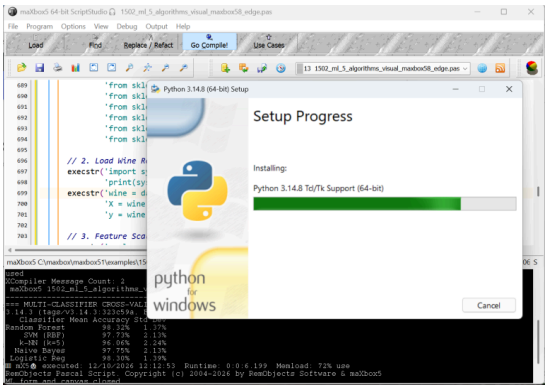
=== MULTI-CLASSIFIER CROSS-
VALIDATION BENCHMARK 22 (WINE DATASET) ===

3.14.3 (tags/v3.14.3:323c59a, Feb 3 2026, 16:04:56) [MSC v.1944 64 bit (AMD64)]

Exception: TclError: Can't find a usable init.tcl in the following directories: *{C:\Users\User\AppData\Local\Programs\Python\Python314\\tcl\tcl8.6}*
This probably means that Tcl wasn't installed properly. at 895.3519
RemObjects Pascal Script. Copyright (c) 2004-
2026 by RemObjects Software & maXbox5

=== MULTI-CLASSIFIER CROSS-
VALIDATION BENCHMARK 22 (WINE DATASET) ===

3.14.8 (tags/v3.14.8:8e6e75d, Sep 30 2026, 18:19:33) [MSC v.1944 64 bit (AMD64)]



3.14.8 (tags/v3.14.8:8e6e75d, Sep 30 2026, 18:19:33) [MSC v.1944 64 bit (AMD64)]

The new script will fusion Pascal, Python and JS (in Edge) in one file:

https://sourceforge.net/projects/maxbox5/files/examples/1502_ml_5_algorithms_visual_maxbox58_edge.pas/download

To point out **False Positives (FP)** and **False Negatives (FN)** on a decision boundary plot in Python and maXbox, you can use **Boolean indexing** in NumPy to filter misclassified points, and then highlight them using custom markers and **plt.annotate()** with **arrowprops**.

---

1. Logical Filtering in Python

For binary classification:

- **False Positives (FP)**: Actual label $y_{true} = 0$, but predicted $\hat{y} = 1$
- **False Negatives (FN)**: Actual label $y_{true} = 1$, but predicted $\hat{y} = 0$

```
# Create boolean masks for errors
fp_mask = (y_test == 0) & (y_pred == 1)
fn_mask = (y_test == 1) & (y_pred == 0)

# True positives and true negatives for reference
tp_mask = (y_test == 1) & (y_pred == 1)
tn_mask = (y_test == 0) & (y_pred == 0)
```

---

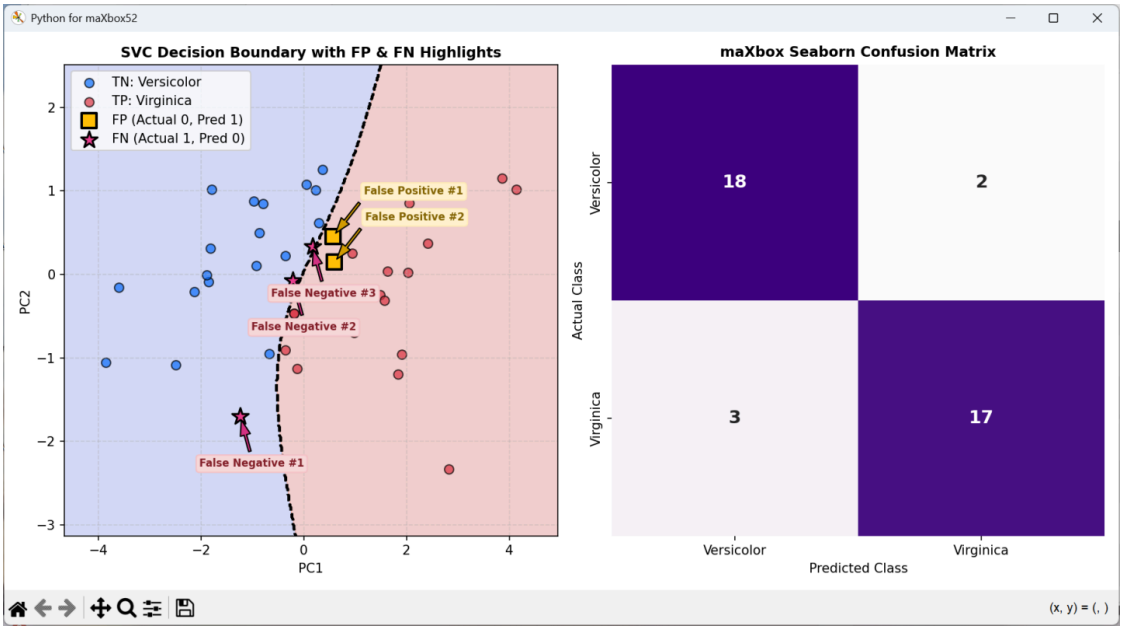
2. Plotting Decision Boundaries & Drawing Arrow Annotations

Using Matplotlib's `plt.annotate()`, you can draw arrows pointing directly to the coordinates of each misclassification:

This updated maXbox Pascal script generates a side-by-side **dual-panel figure**:

1. **Left Panel:** The 2D PCA decision boundary region plot with **arrow annotations pointing directly to False Positives and False Negatives**.
2. **Right Panel:** The matching **Seaborn confusion matrix heatmap**, displaying the exact counts for True Negatives, True Positives, False Positives, and False Negatives.

Would you like to adjust the decision probability threshold (`predict_proba`) to see how shifting the decision boundary trades off False Positives against False Negatives?



https://sourceforge.net/projects/maxbox5/files/examples/1502_ml_6_algorithms_visual_maxbox59_edge.pas/download

```
3.14.8 (tags/v3.14.8:8e6e75d, Sep 30 2026, 18:19:33) [MSC v.1944 64
bit (AMD64)]
              precision recall    f1-score    support

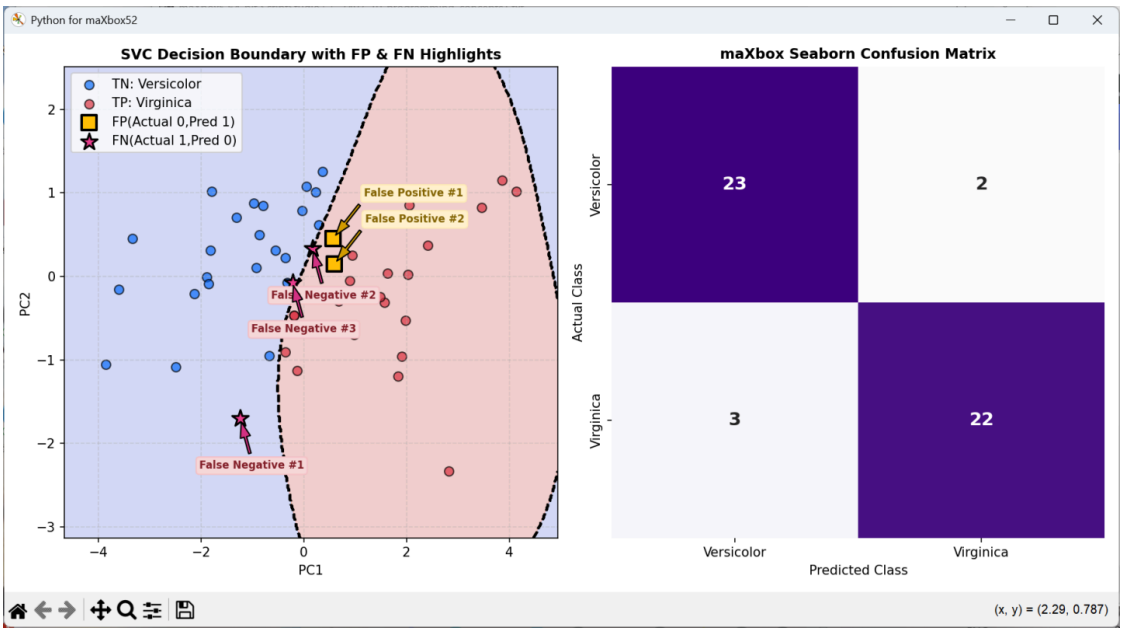
       0       0.86      0.90      0.88        20
       1       0.89      0.85      0.87        20

accuracy                           0.88        40
macro avg 0.88 0.88 0.87 40
weighted avg 0.88 0.88 0.87 40

           Versicolor  Virginica
Versicolor 18          2
Virginica  3           17
 mX5 executed: 12/10/2026 19:37:18 Runtime: 0:0:5.54 Memload: 66%
use
```

Demonstrates how shifting probability threshold trades off False Positives vs False Negatives

https://sourceforge.net/projects/maxbox5/files/examples/1502_ml_6_algorithms_visual_maxbox59_edge.pas/download



https://sourceforge.net/projects/maxbox5/files/examples/1502_ml_6_algorithms_visual_maxbox60_edge.pas/download

	precision	recall	f1-score	support
0	0.88	0.92	0.90	25
1	0.92	0.88	0.90	25
accuracy			0.90	50

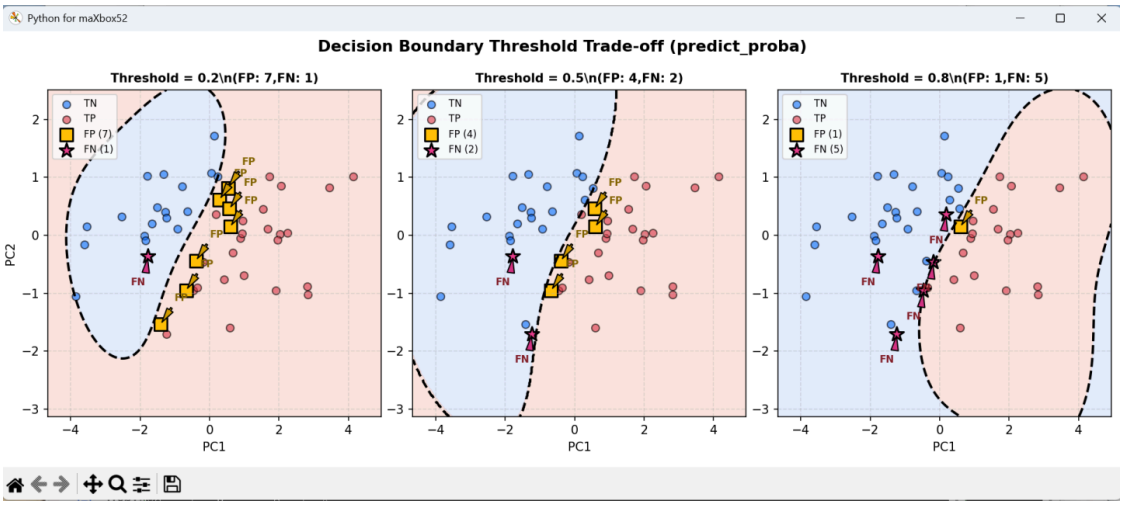
macro avg 0.90 0.90 0.90 50
weighted avg 0.90 0.90 0.90 50

	Versicolor	Virginica
Versicolor	23	2
Virginica	3	22

mX5 executed: 12/10/2026 20:36:17 Runtime: 0:0:52.349 Memload: 82% use

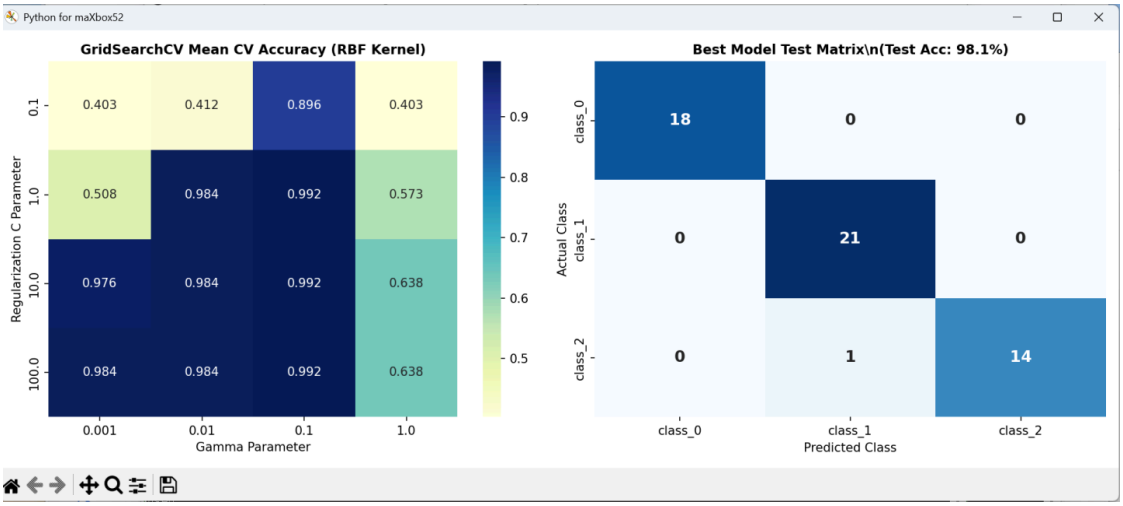
Hyperparameter Threshold Analysis

Demonstrates how shifting probability threshold trades off False Positives vs False Negatives



https://sourceforge.net/projects/maxbox5/files/examples/1502_ml_6_algorithms_visual_maxbox60_edge.pas/download

This script uses scikit-learn's GridSearchCV via TPythonEngine inside maXbox to systematically search across combinations of hyperparameters (such as regularization parameter C, kernel scale γ, and kernel type) using 5-fold stratified cross-validation.



https://sourceforge.net/projects/maxbox5/files/examples/1502_ml_6_algorithms_visual_maxbox61_edge.pas/download

How It Works:

1. Grid Parameter Dictionary (param_grid): Defines a discrete search space of parameters to evaluate across $4 \times 4 \times 2 = 32$ total combinations.
2. Automated Cross-Validation (GridSearchCV): Trains $32 \times 5 = 160$ separate model fits, using 5-fold stratified cross-validation on each combination to find the parameter set with the highest mean validation score.
3. refit=True: Once the best parameters are identified, `GridSearchCV` automatically retrains the final model (`best_estimator_`) on the entire training set.
4. Validation Heatmap (cv_results_ Pivot): Reshapes the cross-validation score history into a 2D matrix (C vs. γ) and plots a Seaborn heatmap so you can visually identify optimal parameter regions and avoid overfitting.

Compare Python4maXbox – Pure Python

```
procedure PY_ConfusionMatrix_Demo5;
begin
  with TPythonEngine.Create(Nil) do begin // [1, 6]
    try
      loadDLL;
      autofinalize:= false; // [6]
      // 1. Import Python Modules
      ExecStr('import pandas as pd' + lf +
              'import matplotlib.pyplot as plt'+lf+
              'import seaborn as sns'+lf+// [1, 2]
              'from sklearn.model_selection import train_test_split
              'from sklearn.ensemble import RandomForestClassifier'
              'from sklearn import datasets'+lf+
              'from sklearn.metrics import accuracy_score, confusio

      // 2. Load dataset & perform train/test split
      execstr('iris = datasets.load_iris()'+lf+
              'df = pd.DataFrame(iris.data, columns=iris.feature_na
              'X_train, X_test, y_train, y_test = train_test_split(
              'df[iris.feature_names],iris.target,test_size=0.5,str

      // 3. Fit Random Forest Classifier
      execstr('rf = RandomForestClassifier(n_estimators=100, random
              'rf.fit(X_train, y_train)'); // [2, 7]

      // 4. Generate Confusion Matrix & Plot Seaborn Heatmap
      execstr('cm = pd.DataFrame(confusion_matrix(y_test, rf.predic
              'columns=iris.target_names, index=iris.target_names)'
              'sns.heatmap(cm, annot=True, cmap="Blues")'+lf+
              'plt.title("maXbox Seaborn Confusion Matrix Plot")'+l
              'plt.xlabel("Predicted")'+lf+
              'plt.ylabel("Actual")'+lf+
              'plt.show()'); // [2, 5]
    except
      raiseError; // [6]
    finally
      Free; // [6]
    end;
  end;
end;
```

Pure Python

1. Pure Python Code (scikit-learn & Seaborn)

```
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
from sklearn import datasets
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matri
from sklearn.model_selection import train_test_split

# 1. Load dataset & train/test split
iris = datasets.load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.5, stratify=iris.target, r
)
```

```
14
15  # 2. Train classifier
16  model = RandomForestClassifier(n_estimators=100, random_state=42)
17  model.fit(X_train, y_train)
18  y_pred = model.predict(X_test)
19
20  # 3. Compute Confusion Matrix DataFrame
21  cm = confusion_matrix(y_test, y_pred)
22  cm_df = pd.DataFrame(
23      cm, index=iris.target_names, columns=iris.target_names
24  )  # [2, 4]
25
26  # 4. Plot Seaborn Heatmap
27  plt.figure(figsize=(6, 5))
28  sns.heatmap(cm_df, annot=True, fmt="d", cmap="Blues")  # [2, 5]
29  plt.title("Confusion Matrix Heatmap")
30  plt.xlabel("Predicted Class")
31  plt.ylabel("Actual Class")
32  plt.tight_layout()
33  plt.show()
34
35  # 5. Print text report
36  print(classification_report(y_test, y_pred, target_names=iris.targ
```

Advertisements

Occasionally, some of your visitors may see an advertisement here, as well as a Privacy & Cookies banner at the bottom of the page. You can hide ads completely by upgrading to one of our paid plans.

UPGRADE NOW DISMISS MESSAGE

Posted in Code, Data Science, Machinelearning, maXbox, Python

4 responses to “Machinelearning Steps”



maxkleiner

December 2, 2025 Edit

Python’s future remains bright because its values align with how developers actually learn and build:

- Readability
- Approachability
- Stability

- A touch of irreverence

As AI continues to influence software development—and Octoverse shows that 80% of new developers on GitHub use GitHub Copilot in their first week—Python's clarity matters more than ever.

Why developers still flock to Python: Guido van Rossum on readability, AI, and the future of programming – The GitHub Blog

★ Like

Reply



maxkleiner

December 19, 2025 Edit

The whiteboard is blank. The marker is uncapped. The interviewer leans back and says design Twitter. And your brain just explodes into this mess of load balancers and microservices and Kafka streams because you think that's what they want to hear. Two minutes in and you're already lost in your own diagram.

★ Like

Reply



maxkleiner

December 28, 2025 Edit

AI will revolutionise everything. It will be the next mega industry. It will beckon in a new, automated industrial revolution. It will separate growth from labour and generate copious amounts of wealth. This is the narrative that has been shoved down our throats by drug-addled techbros and the pathetic financebros that prop them up. This narrative has been used to leverage so much debt and investment into AI that we have basically bet the entire Western economy on its success. But there is one problem: this narrative simply isn't true, and the AI industry is beginning to find that out the hard way.

★ Like

Reply



Python4Delphi Concepts – Trans Europ Express